

Cairo University

Faculty of Computers and Artificial Intelligence



CS251

Introduction to

Software Engineering

Project Name

Software Design Specifications

Version X.X

Team Names and Emails



CS251: <Team Name>

Project: <Project Name>

Software Design Specification

Month & Year

Contents

Instructions [To be removed]	Error! Bookmark not defined.
Team	
3 Document Purpose and Audience	Error! Bookmark not defined.
System Models	
3	
I. Architecture Diagram	3
II. Class Diagram(s)	4
III. Class Descriptions	4
IV. Sequence diagrams	7
Class - Sequence Usage Table	7
V. State Diagram	8
VI. SOLID Principles	8
VII. Design Patterns	9
Tools	
9	



CS251: <Team Name>

Project: <Project Name>

Software Design Specification

Ownership Report

9

Team

ID	Name	Email	Mobile
20240002	Abanoub Emad Makrem	20240002@stud.fci-cu.edu.eg	01274526679
20240696	Yousef Atef Ibrahim	20240696@stud.fci-cu.edu.eg	01500093586
20240088	Alhussien Sayed	20240088@stud.fci-cu.edu.eg	01044880209
20240486	Mohamed Gamal	20240486@stud.fci-cu.edu.eg	0 11 27176816



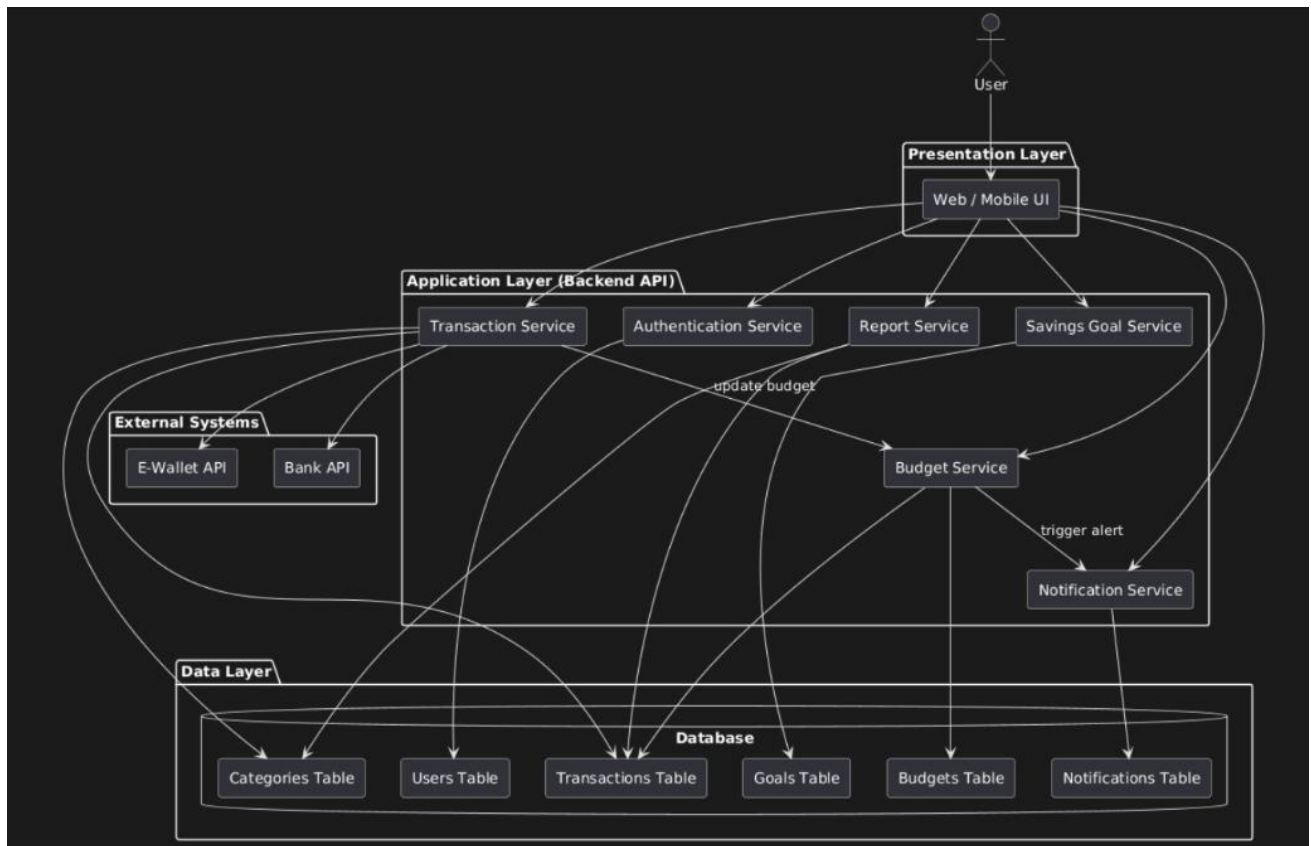
CS251: <Team Name>

Project: <Project Name>

Software Design Specification

System Models

I. Architecture Diagram



II.



CS251: <Team Name>

Project: <Project Name>

Software Design Specification

Category	Entity	Classifies transactions into groups	Shared across budgets (aggregation)
-----------------	--------	-------------------------------------	-------------------------------------

Class Name	type	Responsibility	Keynote
Budget	Entity	Defines spending limits per category and period	Usage is derived from transactions
SavingsGoal	Entity	Tracks user saving targets and progress	Contains target, current amount, deadline
Notification	Entity	Represents system alerts sent to users	Used for budget alerts and system messages
Report	Entity	Represents generated financial reports	Used by reporting service
Session	Entity	Manages user login session lifecycle	Created during authentication
Admin	Entity	Represents admin user with system privileges	Limited scope (management only)
AuthenticationManager	Service	Handles user registration and login	Singleton Pattern



CS251: <Team Name>

Project: <Project Name>

Software Design Specification

TransactionService	Service	Manages adding and retrieving transactions	Triggers budget updates
BudgetService	Service	Handles budget calculations and monitoring	Uses Strategy Pattern for alerts
ReportService	Service	Generates reports and analytics	Reads from transactions/categories

Class Name	type	Responsibility	Keynote
NotificationService	Service	Sends notifications to users	Uses abstraction (DIP)
Transaction Factory	Factory	Creates transaction objects	Encapsulates object creation
AlertStrategy	Interface	Defines alert behavior	Supports extensibility (OCP)
BudgetAlertStrategy	Strategy	Implements budget alert logic	Can be extended later
NotificationSender	Interface	Abstracts notification sending	Enables DIP
EmailSender	Implementation	Sends notifications via email	Implements NotificationSender



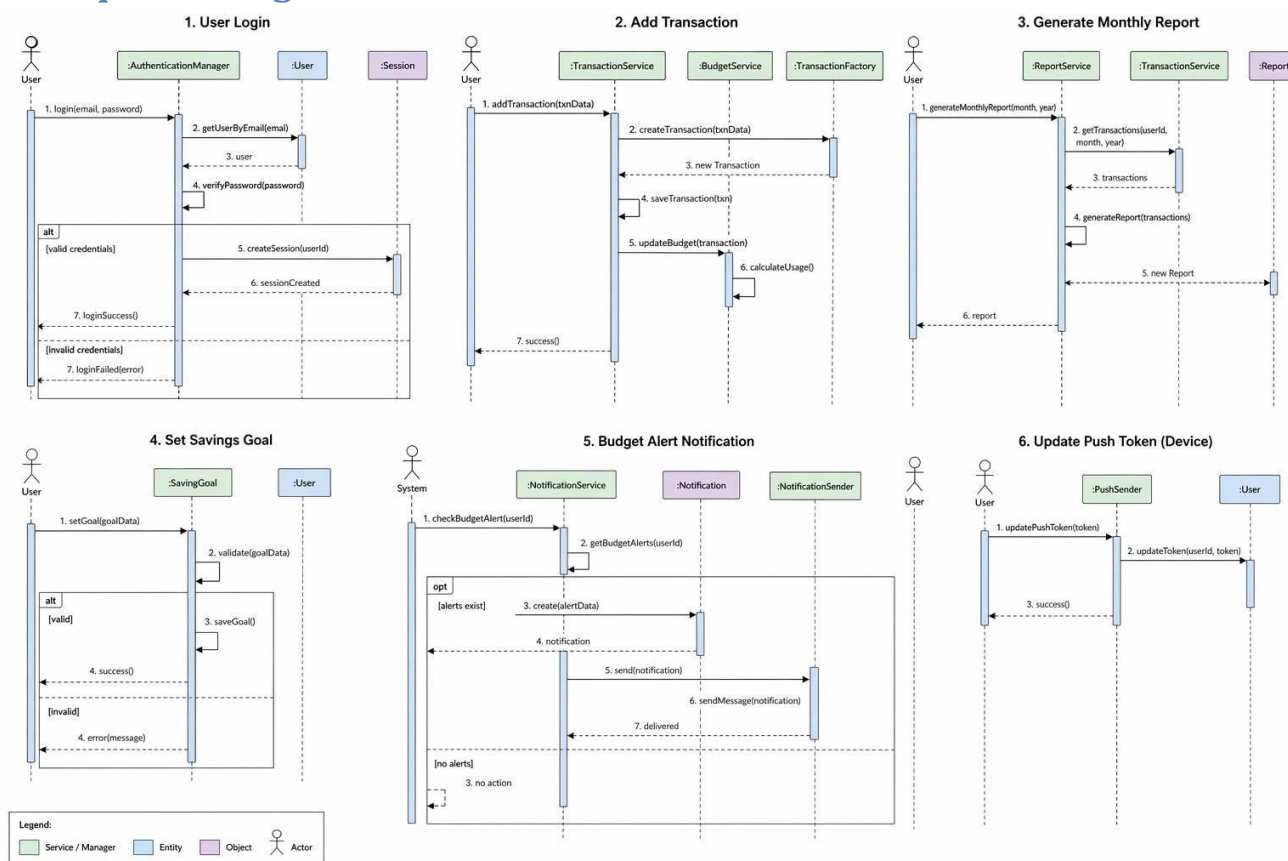
CS251: <Team Name>

Project: <Project Name>

Software Design Specification

PushSender	Impleme ntation	Sends push notifications	Implements NotificationSender
-------------------	--------------------	--------------------------	----------------------------------

IV. Sequence diagrams



Class - Sequence Usage Table

Sequence Diagram	Classes Used	All Methods Used
------------------	--------------	------------------



CS251: <Team Name>

Project: <Project Name>

Software Design Specification

1. User Login	AuthenticationManager User Session	- getUserByEmail(email): User - verifyPassword(password): Boolean - createSession(userId): Session - (entity, returned by getUserByEmail) - (entity, created via createSession)
2. Add Transaction	TransactionService TransactionFactory BudgetService	- addTransaction(txnData): void - saveTransaction(txn): void - createTransaction(txnData): Transaction - updateBudget(transaction: Transaction): void - calculateUsage(): Decimal
3. Generate Monthly Report	ReportService TransactionService Report	- generateMonthlyReport(month, year): Report - generateReport(transactions): Report - getTransactions(userId, month, year): List<Transaction> - (entity, created via generateReport)
4. Set Savings Goal	SavingGoal User	- setGoal(goalData): void - validate(goalData): Boolean - saveGoal(): void - getProgress(): Float - (actor/entity reference)
5. Budget Alert Notification	NotificationService Notification NotificationSender	- checkBudgetAlert(userId): void - getBudgetAlerts(userId): List<Notification> - notify(userId, message: String): void - create(alertData): Notification - send(notification): void - sendMessage(notification): void
6. Update Push Token	PushSender User	- send(message: String): void - updatePushToken(token): void - updateToken(userId, token): void



CS251: <Team Name>

Project: <Project Name>

Software Design Specification

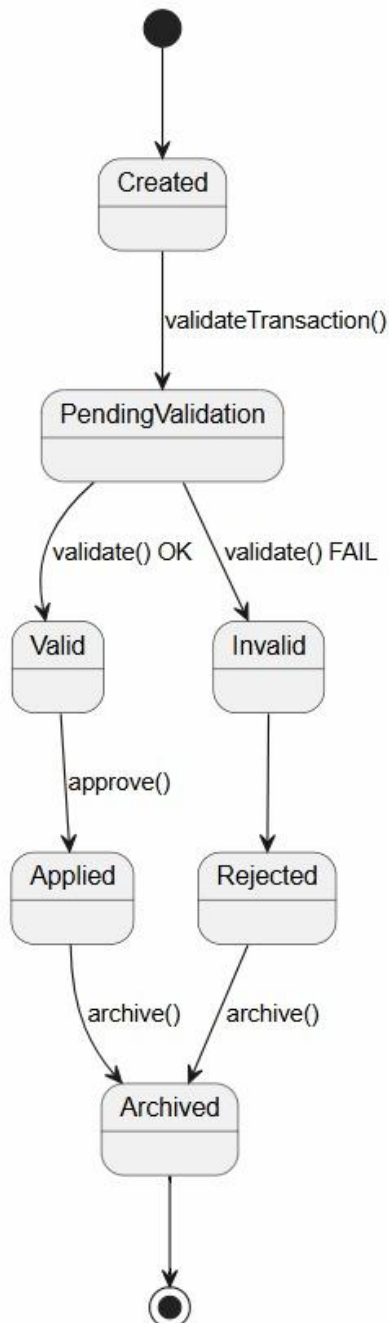


CS251: <Team Name>

Project: <Project Name>

Software Design Specification

V. State Diagram





CS251: <Team Name>

Project: <Project Name>

Software Design Specification

VI. SOLID Principles

Single Responsibility Principle (SRP)

Each class has one responsibility.

Example: PasswordUtil handles passwords only, DatabaseManager handles DB operations only.

Benefit: Easier maintenance and debugging.

Open/Closed Principle (OCP)

New alert behaviors can be added through new strategy classes without changing existing code.

Example: ThresholdAlertStrategy.

Benefit: Easier extension with less risk of breaking code.

Liskov Substitution Principle (LSP)

Any AlertStrategy implementation can replace another safely.

Benefit: Flexible and reliable polymorphism.

Interface Segregation Principle (ISP)

Small focused interfaces like NotificationSender.

Benefit: Classes implement only methods they actually need.

Dependency Inversion Principle (DIP)

High-level classes depend on abstractions, not concrete classes.

Example: depending on NotificationSender instead of UINotificationSender.

Benefit: Lower coupling and easier testing/extending.

VII. Design Patterns

Strategy Pattern

Classes: AlertStrategy, ThresholdAlertStrategy

Used to make alert behavior interchangeable.

Benefit: Easy to add new alert types without modifying existing code.



CS251: <Team Name>

Project: <Project Name>

Software Design Specification

Factory Pattern

Class: TransactionFactory

Used to centralize transaction object creation.

Benefit: Cleaner code and easier extension for new transaction types.

Singleton Pattern (likely)

Class: DatabaseManager

Used to keep one shared database manager instance.

Benefit: Prevents multiple unnecessary DB connections.

Abstraction / Dependency Inversion

Classes: NotificationSender, UINotificationSender

Used to separate notification logic from implementation.

Benefit: Can switch notification methods easily (UI, Email, SMS, etc.).

Utility Class Pattern

Class: PasswordUtil

Used for reusable password helper methods.

Benefit: Avoids duplicated security-related code.

Tools

PlanetUML

Ownership Report

Item	Owners
Abanoub Emad Makrem	Auth, Database, Financial Goals Feature, Documentation, Class Diagram



CS251: <Team Name>

Project: <Project Name>

Software Design Specification

Youssef Atef	Arch Diagram, Transaction, Category.
Alhussien Sayed	Sequence Diagram, fxml UI
Moahmed Gamal	State Diagram,Notofication, Budget Features